AKTU

B.Tech 5th Sem

GATEWAY CLASSES

CS IT & CS Allied

Object Oriented System Design with C++ (BCS054)

Notes

as per AKTU Syllabus

Unit-4

By Amol Sharma sir

- Pursuing Ph.D. from IIT (BHU)
- Wipro Certified Faculty (WCF)

GATEWAY CLASSES

C++ Basics

Overview

- C++ is an object-oriented programming language.
- It was developed by *Bjarne Stroustrup* at AT&T Bell Laboratories in Murray Hill, New Jersey, USA, in the early 1980's.
- Stroustrup, an admirer of Simula67 and a strong supporter of C, wanted to combine the best of both the languages and create a more powerful language that could support object-oriented programming features and still retain the power and elegance of C. The result was C++.
- Therefore, C++ is an extension of C with a major addition of the class construct feature of Simula67. Since the class was a major addition to the original C language, Stroustrup initially called the new language 'C with classes'.
- However, later in 1983, the name was changed to C++.
- The idea of C++ comes from the C increment operator ++, thereby suggesting that C++ is an incremented version of C.
- C+ + is a superset of C. Almost all c programs are also C++ programs. However, there are a few minor differences that will prevent a c program to run under C++ complier.
- The most important facilities that C++ adds on to C care classes, inheritance, function overloading and operator overloading. These features enable creating of abstract data types, inherit properties from existing data types and support polymorphism, thereby making C++ a truly object-oriented language.
- The object oriented features in C++ allow programmers to build large programs with clarity , extensibility and ease of maintenance, incorporating the spirit and efficiency of C. The addition of new features has transformed C from a language that currently facilitates top-down, structured design to one that provides bottom up, object oriented design.

GATEWAY CLASSES

C++ Basics

Overview

Applications:

- C++ is faster than most of the other programming languages and also supports multithreading with concurrency. Thus in Applications where the speed along with concurrency is required, C++ is the most sought-after language for development.
- Apart from speed and performance, C++ is also close to hardware and we can easily manipulate hardware resources using C++ low-level functions. Thus C++ becomes the obvious choice in the applications that require low-level manipulations and hardware programming.

Games

- C++ is close to the hardware, can easily manipulate resources, provide procedural programming over CPU intensive functions and is fast. It is also able to override the complexities of 3D games and provides multilayer networking. All these benefits of C++ make it a primary choice to develop the gaming systems as well as game development suites.
- Examples: Birthright, Hellfire, Football Pro, Bullrider I & II, Trophy Bear, Kings Quest, Antara, Hoyle Card games suite, SWAT, StarCraft: Brood War, Diablo I, Diablo II: Lord of Destruction, Warcraft I & II, World of Warcraft.and too many others to list

GUI Based Applications

- C++ can be used to develop most of the GUI based and desktop applications easily as it has got the required features.

Adobe Systems, Most of the applications of the adobe systems including Illustrator, Photoshop, etc. are developed using C++.

Win Amp Media Player from Microsoft is a popular software that has been catering to all our audio/video needs for decades now. This software is developed in C++.

Database Software

- C++ is also used in writing database management software. The two most popular databases MySQL and Postgres are written in C++.

Operating Systems

- The fact that C++ is a strongly typed and fast programming language makes it an ideal candidate for writing operating systems. In addition to this, C++ has a wide collection of system-level functions that also help in writing low-level programs. Examples: Apple OS and Microsoft Windows OS.

Browsers

- Browsers are mostly used in C++ for rendering purposes. Rendering engines need to be faster in execution as most people do not like to wait for the web page to be loaded. With the fast performance of C++, most browsers have their rendering software written in C++.

Mozilla Firefox

Thunderbird, Just like the Firefox browser, the email client from Mozilla, Thunderbird is also developed in C++. This is also an open-source project.

Google Applications like Google File System and Chrome browser are written in C++.

GATEWAY CLASSES

C++ Basics

Overview

Applications:

Advanced Computation And Graphics

- C++ is useful in developing an application that requires high-performance image processing, real-time physical simulations, and mobile sensor applications that need high performance and speed.

  Maya, 3D software from Alias system is developed in C++ and is used for animation, virtual reality, 3D graphics, and environments.

Banking Applications

- As C++ aids in concurrency, it becomes the default choice for banking applications that require multi-threading, concurrency, and high performance.

  Infosys Finacle, a popular core banking application that uses C++ as backend programming languages.

Cloud/Distributed System

- Cloud storage systems that are extensively used nowadays work close to the hardware. C++ becomes a default choice for implementing such systems as it is close to the hardware. C++ also provides multithreading support that can build concurrent applications and load tolerance.

  Bloomberg, distributed RDBMS application that is used for accurately providing real-time financial information and news to investors. While Bloomberg's RDBMS is written in C, its development environment and set of libraries are written in C++.

Compilers

- Compilers of various high-level programming languages are written either in C or C++. The reason is that both C and C++ are low-level languages that are close to hardware and are able to program and manipulate the underlying hardware resources.Example: GCC

Embedded Systems

- Various embedded systems like smart watches, medical equipment systems use C++ to program as it is closer to the hardware level and can provide a lot of low-level function calls when compared to the other high-level programming languages.

Enterprise Software

- C++ is used in developing many enterprise software as well as advanced applications like flight simulation and radar processing.

Libraries

- When we require very high-level mathematical computations, performance and speed become important. Hence most of the libraries use C++ as their core programming language. Most high-level machine language libraries use C++ as backend.

GATEWAY CLASSES

C++ Basics

A Simple C++ Program

```
//A simple C++ program
#include<iostream>
using namespace std;
/* This program will
be printing a string */
int main()
{
cout<<"c++ is better than c \n";
return 0;
}
```

Program Components:

Include Directive
(iostream.h)
Namespace
(using, std)
Main function
Semi colon
(;)
Comments
(//, /*..*/)
Output Statement
(cout, <<, "")
Return Statement
(return type of main)

GATEWAY CLASSES

C++ Basics

Output Statement:

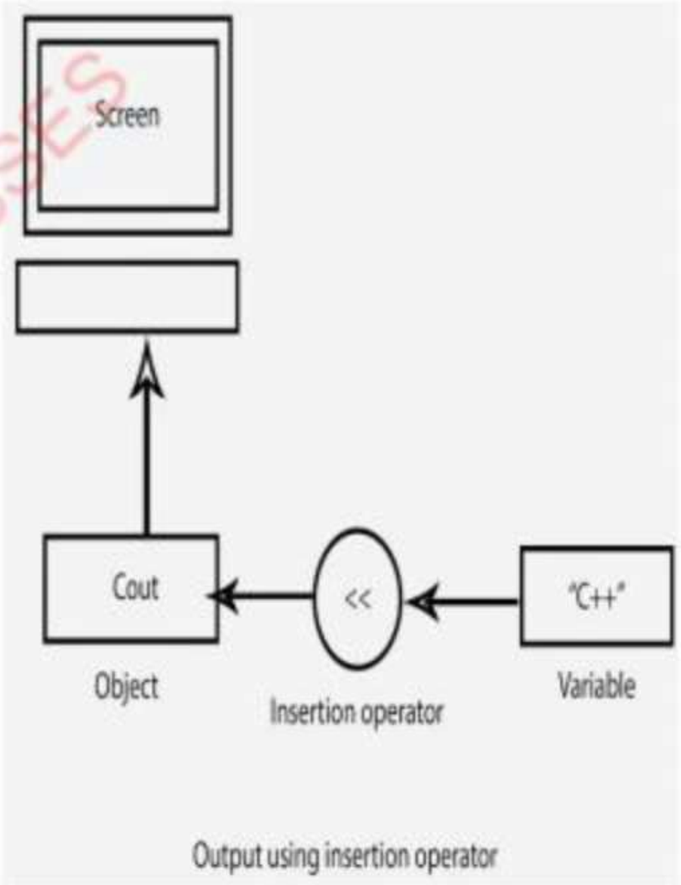
The following is an output statement in C++:

```
cout<<"Hello World";
```

This statement causes the string in quotation marks to be displayed on the screen.

In this statement,

- The identifier cout is a predefined object that represents the standard output stream (i.e. 'Screen') in C++.
- The operator << is called the *insertion or put to operator*. It inserts (or sends) the contents of the variable on its right to the object on its left.



GATEWAY CLASSES

C++ Basics

Input Statement:

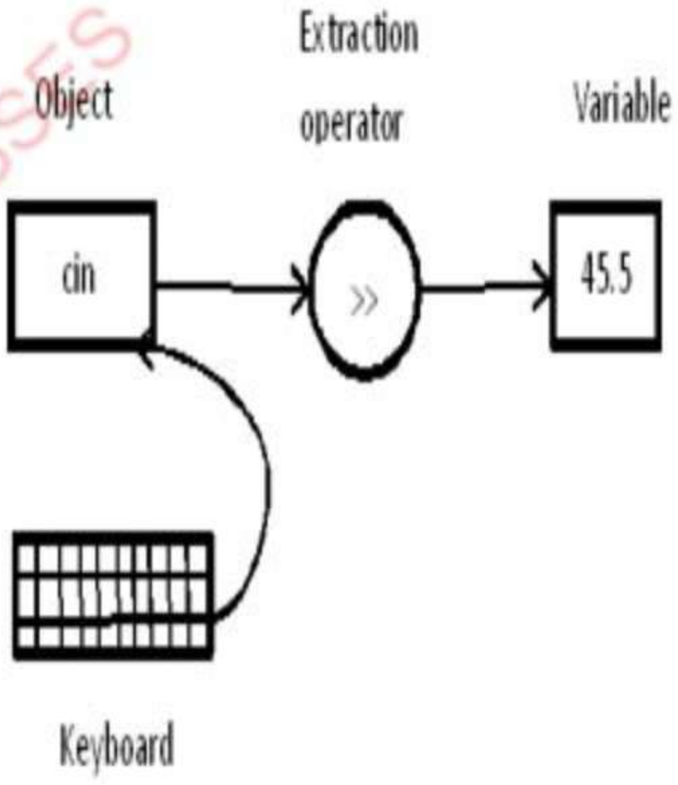
The following is an input statement in C++:

```
cin>>number1;
```

This statement causes the program to wait for the user to type in a number.

In this statement,

- The identifier cin is a predefined object that represents the standard input stream (i.e. 'Keyboard') in C++.
- The operator >> is called the *extraction or get from operator*. It extracts (or takes) the value from the keyboard and assigns it to the variable on its right.



GATEWAY CLASSES

C++ Basics

Cascading of I/O operators:

The multiple use of insertion operator (<<) in output statement and extraction operator (>>) in input statement is called cascading.

The statement

```
cout<<"Result = "<<result<<"\n";
```

First sends the string "Result=" to cout and then sends the value of result and finally it sends the newline character so that the next output will be in the new line.

In the statement

```
cin>>number1>>number2;
```

The values are assigned from left to right i.e. if we key in two values, say 30 and 40, then 30 will be assigned to number1 and 40 to number2.

GATEWAY CLASSES

C++ Basics

Cascading of I/O operators:

Example:

sum.cpp

```
1   #include<iostream>
2   using namespace std;
3   int main()
4   {
5       int number1, number2;
6       int result;
7
8       cout<<"Enter two numbers: ";
9       cin>>number1>>number2;
10
11      result=number1+number2;
12
13      cout<<"Result= "<<result;
14      return 0;
15  }
16
```

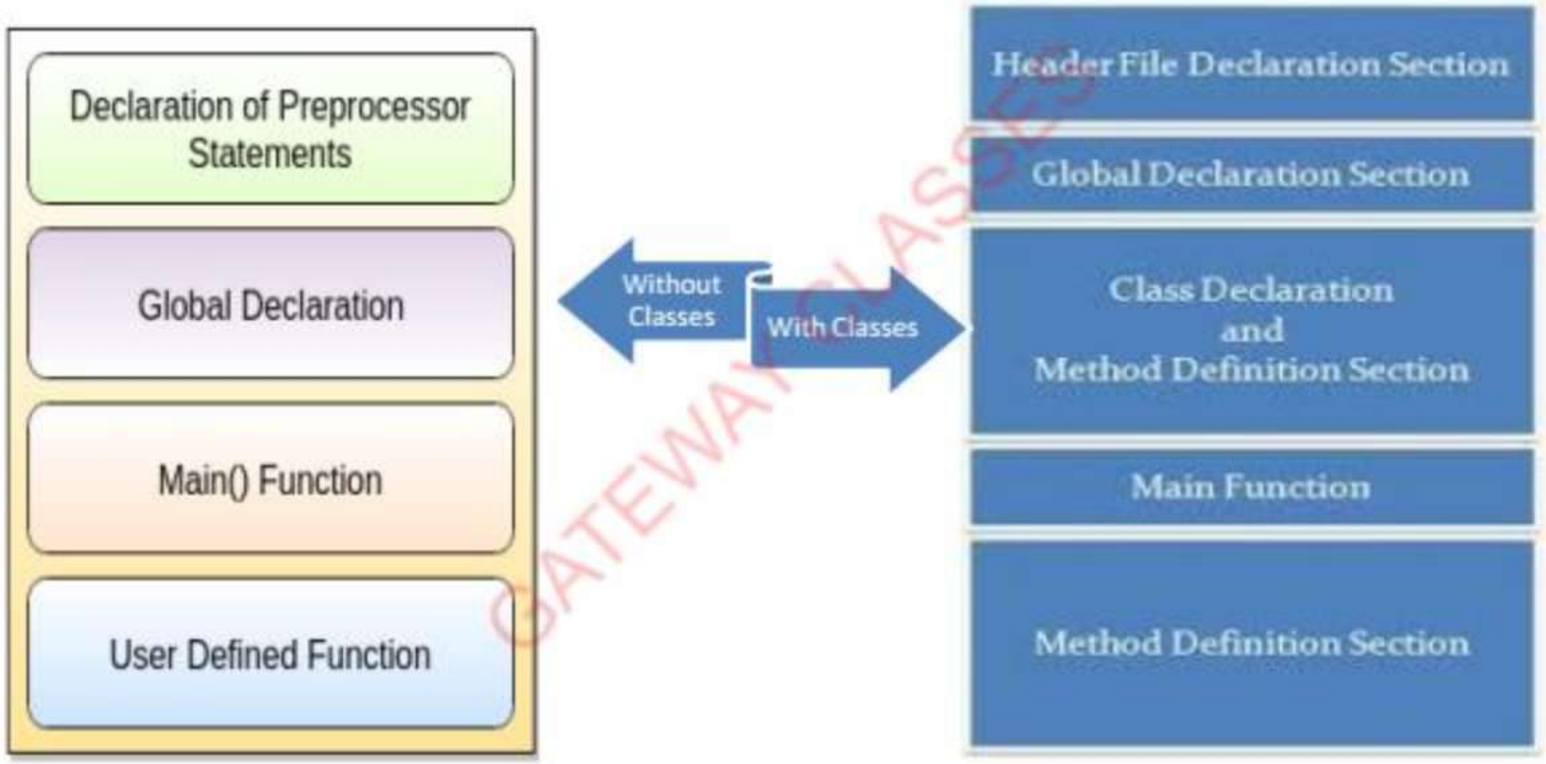
D:\docs\C++\sum.exe

```
Enter two numbers: 30
40
Result= 70
--------------------------------
Process exited after 4.635 seconds with return value 0
Press any key to continue . . .
```

GATEWAY CLASSES

C++ Basics

Program Structure



GATEWAY CLASSES

C++ Basics

Identifiers:

"Identifiers refer to the names of variables, functions, arrays, classes, etc., created by the programmer."

Rules for naming identifiers:

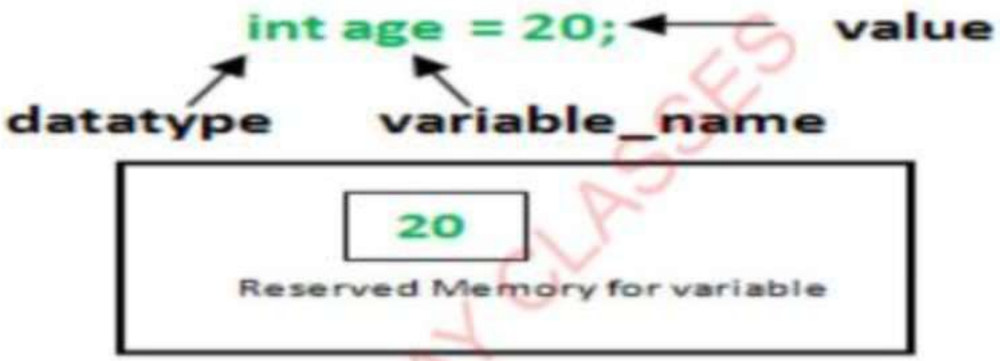
1. Only alphabetic characters, digits and underscores are permitted.
2. The name cannot start with a digit.
3. Uppercase and lowercase letters are distinct.
4. A declared keyword cannot be used as a variable name.
5. An identifier can have any number of characters. C++ places no limit on the length of the identifier.

GATEWAY CLASSES

C++ Basics

Variables:

"A variable is a name given to a memory location. It is the basic unit of storage in a program."



GATEWAY CLASSES

- The value stored in a variable can be changed during program execution.
- A variable is only a name given to a memory location, all the operations done on the variable effects that memory location.
- In C++, all the variables must be declared before use.
- C++ allows the declaration of a variable anywhere in the scope. This means that a variable can be declared right at the place of its first use.
- In C++, a variable can be initialized at run time using expressions at the place of declaration. This is called *dynamic initialization*.

float average=sum/i;

C++ Basics

Reference Variables:

"A reference variable provides an alias (alternative name) for a previously defined variable."

```
float total = 50;
float & sum = total;
```

Here variable sum is variable acting as a reference to variable total, sum and total can now be used interchangeably.

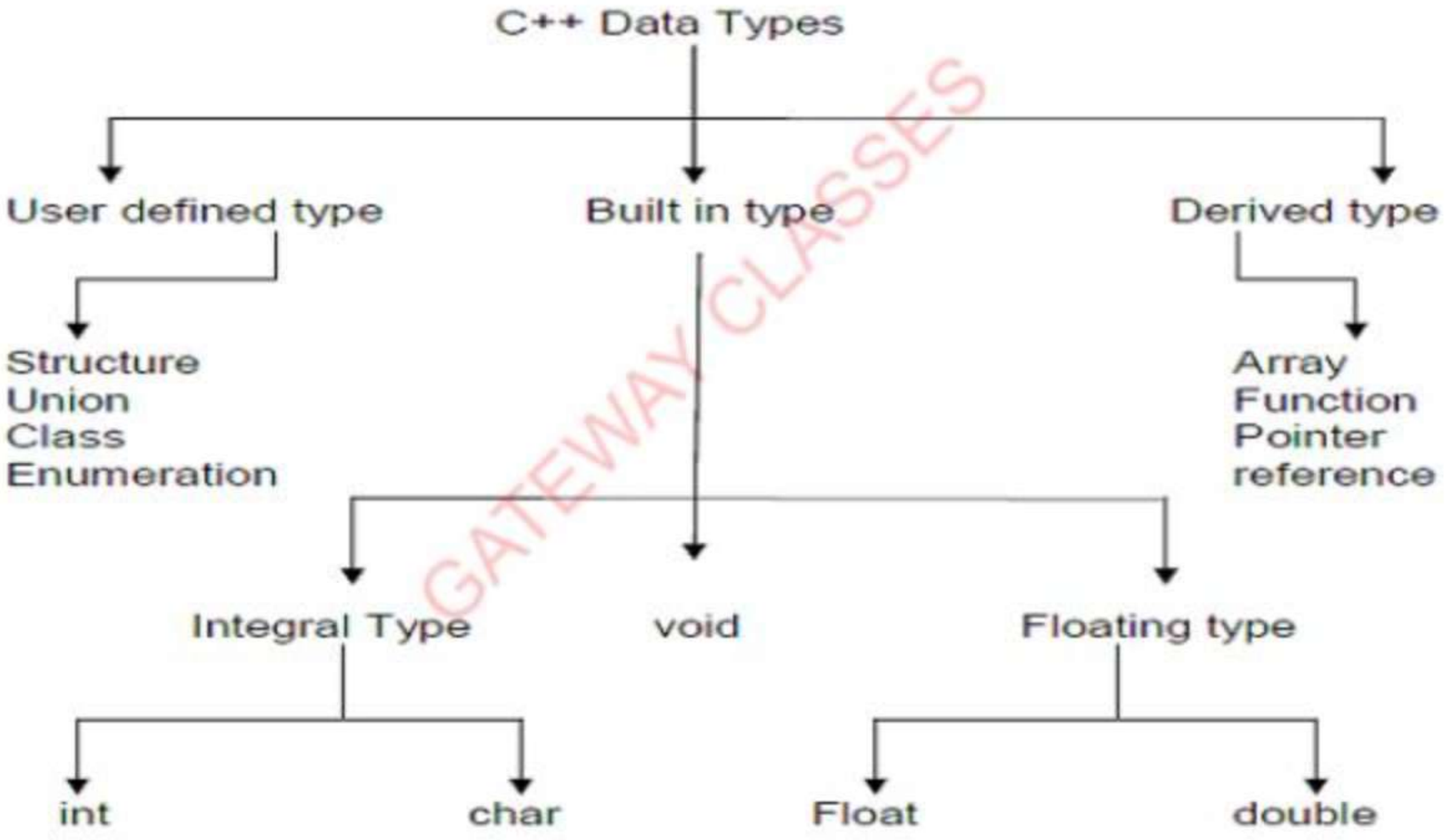
Note:

- *A reference variable must be initialized at the time of its declaration.*
- *Here, & is not address of operator.*

GATEWAY CLASSES

C++ Basics

Data Types:



GATEWAY CLASSES

C++ Basics

Constants:

"Constants refer to fixed values that do not change during the execution of a program."

- Literal Constant:

A literal constant, or simply a literal, is a value, such as a number, character, or string that may be assigned to a variable or symbolic constant, used as an operand in an arithmetic or logical operation, or as a parameter to a function.

Literals are hardcoded values, C++ supports several kinds of literal constants such as the integer 5, floating point number 12.34, the character 'A', or the string, "Hello, world!" etc.

GATEWAY CLASSES

C++ Basics

Constants:

- Symbolic Constant:

  A symbolic constant is a constant with an identifier (label) (name) whose value never changes during the course of a program.

  There are two ways of creating symbolic constants in C++:

1. *Using the qualifier const, as const int symConst=12;*
   - *C++ requires const to be initialized.*
2. *Defining a set of integer constants using enum keyword.*

GATEWAY CLASSES

C++ Basics

Enumerated Data Type (enum)

"Enumerated data type is a user defined type which provides a way for attaching names to numbers."

This facility provides alternative means for creating symbolic constants.

Example:

```
enum colour {red, blue, green};
```

The tag name colour is a type name which can be used to declare variables of this type.

```
colour frontClr;
```

- By default, the enumerators are assigned integer values starting with 0 and then 1 and so on.
- We can override the default by explicitly assigning integer values to the enumerators.

```
enum colour {red, blue=4, green=6}
```

→ red is 0 here

```
enum colour { red=5, blue, green}
```

→ Blue is 6 and green is 7 here

- C++ also permits the creation of anonymous enums (enums without name).

```
enum {on, off}
```

- Each enumerated data type retains its own separate type. This means that int value can't be converted to enum value automatically.

```
colour frontClr=9; //Error
```

- However, an enumerated value can be used in place of an int value;

```
int var=red;
```

GATEWAY CLASSES

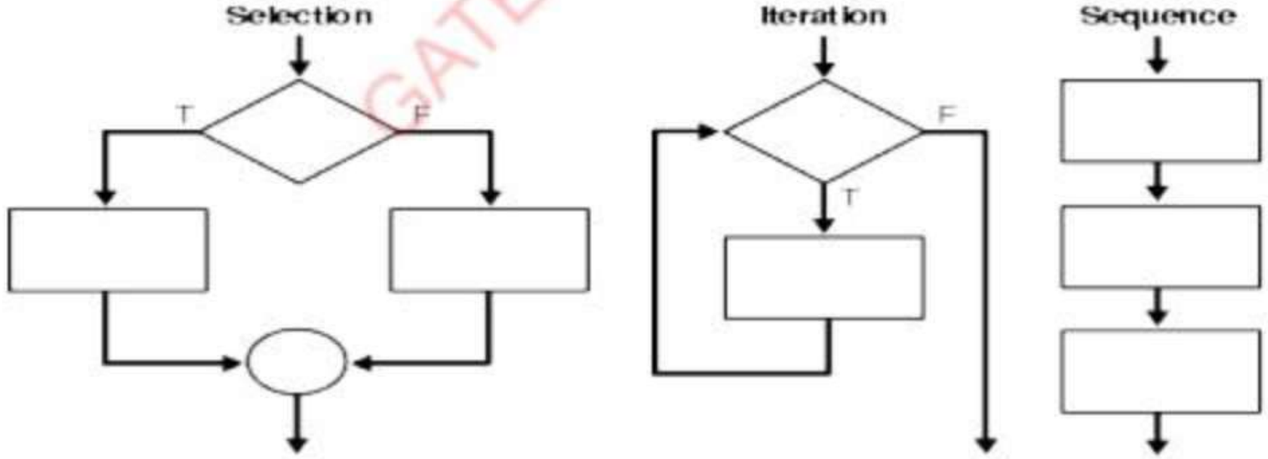
C++ Basics

Control Structures

A program is usually not limited to a linear sequence of instructions. During its process it may bifurcate, repeat code or take decisions. For that purpose, C++ provides control structures that serve to specify what has to be done by our program, when and under which circumstances.

1. *Sequence Structure*
2. *Selection (Decision) (Conditional) Structure*
3. *Iteration (Repetition) (Loop) Structure*

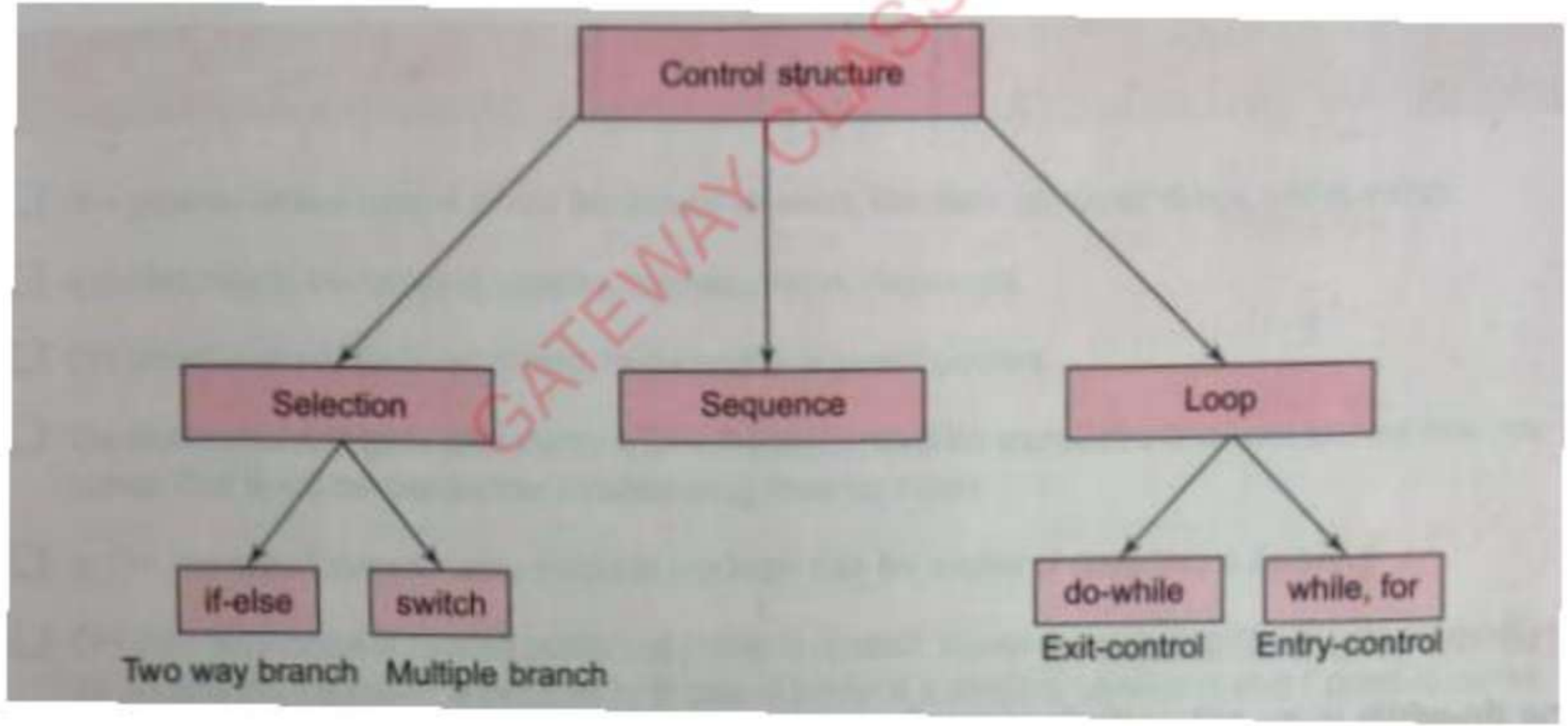
GATEWAY CLASSES



C++ Basics

Control Structures

C++ supports all the three basic control structures, and implements them using various control statements as shown below:



GATEWAY CLASSES

C++ Basics

Control Structures

C++ supports all the three basic control structures, and implements them using various control statements as shown below:

- if (condition) statement
- if (condition) statement1 else statement2
- switch (expression) {

  case constant1: group of statements 1;

  break;

  case constant2: group of statements 2;

  break; . . .

  default: default group of statements }
- while (expression) statement;
- do statement while (condition);
- for (initialization; condition; inc/ dec) statement;

GATEWAY CLASSES

C++ Basics

Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C++ is rich in built-in operators and provide the following types of operators –

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Misc Other Operators

GATEWAY CLASSES

C++ Basics

Operators

Level	Precedence group	Operator	Description	Associativity
1	Scope	::	scope qualifier	Left-to-right
2	Postfix (unary)	++ --	decrement	Left-to-right
		()	functional forms	
		[]	subscript	
		. ->	member access	
3	Prefix (unary)	++ --	decrement	Right-to-left
		~ !	NOT	
		+ -	unary prefix	
		& *	reference / dereference	
		new delete	allocation / deallocation	
		sizeof	parameter pack	
		(*type*)	C-style type-casting	
4	Pointer-to-member	.* ->*	access pointer	Left-to-right
5	Arithmetic: scaling	* / %	multiply, divide, modulo	Left-to-right
6	Arithmetic: addition	+ -	addition, subtraction	Left-to-right
7	Bitwise shift	<< >>	shift left, shift right	Left-to-right
8	Relational	< > <= >=	comparison operators	Left-to-right
9	Equality	== !=	equality / inequality	Left-to-right
10	And	&	bitwise AND	Left-to-right
11	Exclusive or	^	bitwise XOR	Left-to-right
12	Inclusive or	|	bitwise OR	Left-to-right
13	Conjunction	&&	logical AND	Left-to-right
14	Disjunction	||	logical OR	Left-to-right
15	Assignment-level expressions	= *= /= %= += -= >>= <<= &= ^= |=	assignment / compound assignment	Right-to-left
		?:	conditional operator	
16	Sequencing	,	comma separator	Left-to-right

GATEWAY CLASSES

C++ Basics

Type Conversion

C++ permits *explicit type conversion* of variables or expressions using the type cast operator.

Traditional C casts are augmented in C++ by a function call notation as a syntactic alternative

Example:

```
int i;
float f = 3.14;
i = (int) f;
```

Or

```
i = int (f);
```

Wherever data types are mixed in an expression, C++ performs the conversions automatically. This process is known as *implicit conversion*.

If the operand type differ, the compiler converts one of them to match with the other, using the rule that the smaller type is converted to the wider type.

Example:

```
a=3+5.6
    // 3 will be converted to float implicitly.
```

GATEWAY CLASSES

C++ Functions

INTRODUCTION

Functions are the building blocks of C++ programs where all the program activity occurs. Function is a collection of declarations and statements.

Need for a Function

Monolithic program (a large single list of instructions) becomes difficult to understand. For this reason functions are used. A function has a clearly defined objective (purpose) and a clearly defined interface with other functions in the program. Reduction in program size is another reason for using functions. The functions code is stored in only one place in memory, even though it may be executed as many times as a user needs.

The following program illustrates the use of a function:

```
//to display general message using function
#include<iostream>
using namespace std;
void disp(); //function prototype
int main()
{
      disp(); //function call
      return 0;
}
//function definition
void disp()
{
      cout<<"Welcome to the GJU of S&T\n";
      cout<<"Programming is nothing but logic implementation";
}
```

GATEWAY CLASSES

1.1 FUNCTION DEFINITION AND DECLARATION

In C++, a function must be defined prior to it's use in the program. The function definition contains the code for the function. The general syntax of a function definition in C++ is shown below:

```
Type name_of_the_function (argument list)
{
//body of the function
}
```

Here, the type specifies the type of the value to be returned by the function. It may be any valid C++ data type. When no type is given, then the compiler returns an integer value from the function.

Name_of_the_function is a valid C++ identifier (no reserved word allowed) defined by the user and it can be used by other functions for calling this function.

Argument list is a comma separated list of variables of a function through which the function may receive data or send data when called from other function. When no parameters, the argument list is empty. The following function illustrates the concept of function definition:

```
//function definition add()
void add()
{
      int a,b,sum;
      cout<<"Enter two integers"<<endl;
      cin>>a>>b;
      sum=a+b;
      cout<<"\nThe sum of two numbers is "<<sum<<endl;
}
```

The above function add ( ) can also be coded with the help of arguments of parameters as shown below:

```
//function definition add()
void add(int a, int b) //variable names are must in definition
{
      int sum;
      sum=a+b;
      cout<<"\nThe sum of two numbers is "<<sum<<endl;
}
```

1.2 ARGUMENTS TO A FUNCTION

Arguments(s) of a function is (are) the data that the function receives when called/invoked from another function.

1.2.1 PASSING ARGUMENTS TO A FUNCTION

It is not always necessary for a function to have arguments or parameters. The following example illustrates the concept of passing arguments to function **SUMFUN ( ):**

```
// demonstration of passing arguments to a function
#include<iostream>
using namespace std;

float SUMFUN(float x, int N); //function declaration

int main ()
{
      float x,result; //local variables
      int N;
      //formal parameters, Semicolon here, return type
      result = SUMFUN(X,N); //function call
      return 0;
}
//function SUMFUN() definition, No semicolon here
float SUMFUN(float x,int N) //function declaration
{
      ......................
      ..................... Body of the function
      ......................
}
```

1.2.2 DEFAULT ARGUMENTS

GATEWAY CLASSES

C++ allows a function to assign a parameter the default value in case no argument for that parameter is specified in the function call.

For example.

```
// demonstrate default arguments function
#include<iostream>
using namespace std;
int calc(int u)
{
      if (u % 2 == 0)
            return u+10;
      else
            return u+2;
}
void pattern (char M, int B=2)
{
      for (int CNT=0;CNT<B; CNT++)
      {
            cout<<calc(CNT) <<M;
      }
      cout<<endl;
}
int main ()
{
      pattern('*');
      pattern ('#',4);
      pattern ('@',3);
      return 0;
}
```

1.2.3 CONSTANT ARGUMENTS

A C++ function may have constant arguments(s). These arguments(s) is/are treated as constant(s). These values cannot be modified by the function.

For making the arguments(s) constant to a function, we should use the keyword **const** as given below in the function prototype :

```
void max(const float x, const float y, const float z);
```

Here, the qualifier **const** informs the compiler that the arguments(s) having **const** should not be modified by the function max (). These are quite useful when call by reference method is used for passing arguments.

1.3 CALLING FUNCTIONS

In C++ programs, functions with arguments can be invoked by:

(a) Value
(b) *Reference*

GATEWAY CLASSES

Call by Value: In this method the values of the actual parameters (appearing in the function call) are copied into the formal parameters (appearing in the function definition), i.e., the function creates its own copy of argument values and operates on them. The following program illustrates this concept:

```
//calculation of compound interest using a function
#include<iostream>
#include<math.h> //for pow()function
using namespace std;

void calculate (float, float, float); //function prototype
int main()
{
      float principal, rate, time; //local variables

      cout<<"\nEnter the following values:\n";
      cout<<"\nPrincipal:";
      cin>>principal;
      cout<<"\nRate of interest:";
      cin>>rate;
      cout<<"\nTime period (in yeaers) :";
      cin>>time;
      calculate (principal, rate, time); //function call
      return 0;
}
//function definition calculate()
void calculate (float p, float r, float t)
{
      float interest; //local variable
      interest = p* (pow((1+r/100.0),t))-p;
      cout<<"\nCompound interest is : "<<interest;
}
```

Call by Reference: A reference provides an alias – an alternate name – for the variable, i.e., the same variable's value can be used by two different names : the original name and the alias name.
In call by reference method, a reference to the actual arguments(s) in the calling program is passed (only variables). So the called function does not create its own copy of original value(s) but works with the original value(s) with different name. Any change in the original data in the called function gets reflected back to the calling function.

It is useful when you want to change the original variables in the calling function by the called function.

```
//Swapping of two numbers using function call by reference
#include<iostream>
using namespace std;
void swap (int &, int &); //function prototype
int main()
{
      int num1,num2;
      cin>>num1>>num2;

      cout<<"\nBefore swapping:\nNum1: "<<num1;
      cout<<endl<<"num2: "<<num2;
```

GATE DAY CLASSES

```
        swap(num1,num2); //function call

        cout<<"\n\nAfter swapping : \nNum1: "<<num1;
        cout<<endl<<"num2: "<<num2;
        return 0;
}
//function definition swap()
void swap (int & a, int & b)
{
        int temp=a;
        a=b;
        b=temp;
}
```

1.4 RETURN BY REFERENCE

A function can also return a reference. Consider the following program:

```
#include<iostream>
using namespace std;

int& max(int &,int &);

int main()
{
        int x=4,y=9;
        int val=23;

        cout<<"x= "<<x<<endl;
        cout<<"y= "<<y<<endl;

        max(x,y)=val;

        cout<<setw(10)<<"x= "<<x<<endl;
        cout<<setw(10)<<"y= "<<y<<endl;

        //cout<<"Maximum = "<<maximum<<endl;

        return 0;
}
int& max(int &a, int &b)
{
        if(a>b)
                return a;
        else
                return b;
}
```

GATEWAY CLASSES

Since the return type of max() is int&, the function returns reference to a or b (and not the values). Then a function call such as max(x,y) will yield a reference to either x or y depending on their values. This

means that this function call can appear on the left hand side of an assignment statement. That is, the statement

max(x,y)=val;

is legal and assigns 23 to x if it is larger, otherwise 23 to b.

1.5 INLINE FUNCTIONS

These are the functions designed to speed up program execution. An inline function is expanded (i.e. the function code is replaced when a call to the inline function is made) in the line where it is invoked. You are familiar with the fact that in case of normal functions, the compiler has to jump to another location for the execution of the function and then the control is returned back to the instruction immediately after the function call statement. So execution time taken is more in case of normal functions. There is a memory penalty in the case of an inline function.

The system of inline function is as follows :

```
inline function_header

{

      body of the function

}

For example,

#include<iostream>
using namespace std;

//function definition min()
inline void min (int x, int y)
{
      cout<< (x < y? x : y);
}
int main()
{
      int num1, num2;
      cout<<"\Enter the two intergers\n";
      cin>>num1>>num2;
      min (num1,num2); //function code inserted here
      return 0;
}
```

An inline function definition must be defined before being invoked as shown in the above example. Here min ( ) being inline will not be called during execution, but its code would be inserted into main ( ) as shown and then it would be compiled.

If the size of the inline function is large then heavy memory pentaly makes it not so useful and in that case normal function use is more useful.

GATEWAY CLASSES

The inlining does not work for the following situations :

1. For functions returning values and having a *loop* or a *switch* or a goto statement.
2. For functions that do not return value and having a return statement.
3. For functions having static variable(s).
4. If the inline functions are recursive (i.e. a function defined in terms of itself).

The benefits of inline functions are as follows :

1. Better than a macro.
2. Function call overheads are eliminated.
3. Program becomes more readable.
4. Program executes more efficiently.

Difference between Inline Function and Macro in C++ :

S.NO	INLINE FUNCTION	MACRO
1	An inline function is defined by the **inline** keyword.	Whereas the macros are defined by the **#define** keyword.
2	Through inline function, the class's data members can be accessed.	Whereas macro can't access the class's data members.
3	In the case of inline function, the program can be easily debugged.	Whereas in the case of macros, the program can't be easily debugged.
4	In the case of inline, the arguments are evaluated only once.	Whereas in the case of macro, the arguments are evaluated every time whenever macro is used in the program.
5	In C++, inline may be defined either inside the class or outside the class.	Whereas the macro is all the time defined at the beginning of the program.
6	In C++, inside the class, the short length functions are automatically made the inline functions.	While the macro is specifically defined.
7	Inline is not as widely used as macros.	While the macro is widely used.
8	Inline is not used in competitive programming.	While the macro is very much used in competitive programming.
9	Inline function is terminated by the curly brace at the end.	While the macro is not terminated by any symbol, it is terminated by a new line.

GATEWAY CLASSES

1.6 FUNCTION OVERLOADING:

Overloading refers to the use of the same thing for different purposes. C++ also permits overloading functions .This means that we can use the same function name to create functions that perform a variety of different tasks. This is known as function polymorphism in oops.
Using the concepts of function overloading, a family of functions with one function name but with different argument lists in the functions call .The correct function to be invoked is determined by checking the number and type of the arguments but not on the function type.

For example an overloaded add() function handles different types of data as shown below:

```
//Declaration
int add(int a, int b); //prototype 1
int add (int a, int b, int c); //prototype 2
double add(double x, double y); //prototype 3
double add(double p , double q); //prototype 4

//function call
cout<<add(5,10); //uses prototype 1
cout<<add(15,10.0); //uses prototype 4
cout<<add(12.5,7.5); //uses prototype 3
cout<<add(5,10,15); //uses prototype 2
cout<<add(0.75,5); //uses prototype 5
```

GATEWAY CLASSES

A function call first matches the prototype having the same no and type of arguments and then calls the appropriate function for execution.
The function selection invokes the following steps:-

a) The compiler first tries to find an exact match in which the types of actual arguments are the same and use that function.

b) If an exact match is not found the compiler uses the integral promotions to the actual arguments such as :
   char to int
   float to double
   to find a match

c) When either of them fails, the compiler tries to use the built in conversions to the actual arguments and them uses the function whose match is unique. If the conversion is possible to have multiple matches, then the compiler will give error message.

Example:
long square (long n);
double square(double x);
A function call such as :- square(10)

Will cause an error because int argument can be converted to either long or double .There by creating an ambiguous situation as to which version of square( )should be used.

Example Program:

```
#include<iostream>
using namespace std;

int volume(int);
double volume( double , int );
double volume(int ,int ,int);

int main( )
{
      cout<<volume(10)<<endl;
      cout<<volume(2.5,10)<<endl;
      cout<<volume(2,3,6)<<endl;

      return 0;
}
int volume(int s)
{
      return (s*s*s); //cube
}
double volume(double r, int h)
{
      return(3.1416*r*r*h); //cylinder
}
double volume (int l, int b, int h)
{
      return(l*b*h); //cuboid
}
```

GATEWAY CLASSES

Thank you

GATEWAY CLASSES